

src/controller.py

280 lines

```
1 """
2 controller.py
3 =====
4
5 Brute-force Model Predictive Controller (MPC) for the production choke.
6
7 At every control interval (Ts = 1 hour) the controller:
8
9 1. PREDICT - for every candidate next choke position (current +/- up to
10 the ramp-rate limit, on a fine grid), roll the identified
11 `WellModel` forward over a short prediction horizon,
12 holding the candidate constant for the rest of the horizon
13 (move-blocking). Every prediction starts from the CURRENT
14 MEASURED state, so model error never compounds across
15 control intervals - only within one horizon.
16
17 2. FILTER - reject any candidate whose predicted WHP, FLP or BHP would
18 breach the safe operating envelope at ANY point in the
19 horizon. A small safety margin is enforced *inside* every
20 hard limit so sensor noise and plant/model mismatch can
21 never push the real well over the actual line.
22
23 3. SELECT - among the surviving (safe) candidates, choose the one whose
24 predicted end-of-horizon flow rate is closest to the target,
25 with a small penalty on move size for smoothness. Ties are
26 broken deterministically: smallest |move|, then the lower
27 choke position (the more conservative choice).
28
29 4. FALLBACK - if every candidate is predicted to violate a constraint
30 (e.g. a very aggressive starting condition), the controller
31 does not throw up its hands: it picks the candidate that
32 minimises the worst predicted violation magnitude, which is
33 always the "hold or close slightly" direction, and is
34 incapable of making things worse.
35
36 This is exactly why the controller cannot oscillate on an infeasible target:
37 moving further open is rejected outright by step 2 once it would cross a
38 limit within the horizon, and moving back is never selected because it is
39 farther from (the unreachable) target while offering no constraint benefit -
40 so the cost function pins the choke at the largest FEASIBLE opening.
41
42 A steady-state bias/disturbance estimate (`_update_bias`) is maintained per
43 output: at every interval the previous prediction is compared with the new
44 measurement, and the small difference is added to all future steady-state
45 targets. This removes any steady-state offset caused by the model being an
46 approximation of the true plant, which is what lets the controller land
47 exactly on target rather than merely near it.
48 """
49
50 from __future__ import annotations
51
52 from dataclasses import dataclass, field
53 from typing import Dict, List, Optional
54
55 import numpy as np
56
57 from model import WellModel, OUTPUTS
58
59 try:
60     from .config import ControllerParams, Limits, CONTROLLER, LIMITS
61 except ImportError: # pragma: no cover
62     from config import ControllerParams, Limits, CONTROLLER, LIMITS
63
64
65 @dataclass
66 class CandidateResult:
67     """Full record of one evaluated candidate, kept for the dashboard's
```

```

68 'reasoning' panel and for logging."""
69
70 u: float
71 du: float
72 feasible: bool
73 reject_reason: Optional[str]
74 predicted_Q_end: float
75 predicted_trajectory: Dict[str, np.ndarray]
76 cost: float
77 max_violation: float # 0 if feasible; otherwise worst overshoot
78
79
80 @dataclass
81 class ControllerDecision:
82     """Everything the controller decided at one control interval."""
83
84     u_selected: float
85     u_previous: float
86     target_Q: float
87     candidates: List[CandidateResult]
88     selected_index: int
89     all_infeasible: bool
90     disturbance: Dict[str, float]
91
92
93 class ChokeMPC:
94     """Brute-force MPC for the single-well production choke."""
95
96     def __init__(
97         self,
98         model: WellModel,
99         limits: Limits = LIMITS,
100         params: ControllerParams = CONTROLLER,
101     ) -> None:
102         self.model = model
103         self.limits = limits
104         self.p = params
105         # Online disturbance estimate (plant - model) per output, used to
106         # remove steady-state offset. Starts at zero (no bias assumed).
107         self.disturbance: Dict[str, float] = {k: 0.0 for k in OUTPUTS}
108         self._last_prediction: Optional[Dict[str, float]] = None
109         # Filtered measurement state (see ControllerParams.meas_filter_alpha).
110         self._y_filt: Optional[Dict[str, float]] = None
111
112     def _filter_measurement(self, measured: Dict[str, float]) -> Dict[str, float]:
113         """First-order filter on the incoming measurement, so that sensor noise
114         does not randomly flip candidates across the constraint boundary."""
115         a = self.p.meas_filter_alpha
116         if self._y_filt is None:
117             self._y_filt = {k: float(measured[k]) for k in OUTPUTS}
118         else:
119             for k in OUTPUTS:
120                 self._y_filt[k] = (1 - a) * self._y_filt[k] + a * float(measured[k])
121             return dict(self._y_filt)
122
123     # -----
124     def _update_bias(self, measured: Dict[str, float]) -> None:
125         """Compare the previous step's one-step-ahead prediction with the new
126         measurement and nudge the disturbance estimate (exponential filter)."""
127         if self._last_prediction is None:
128             return
129         a = self.p.bias_filter_alpha
130         for k in OUTPUTS:
131             err = measured[k] - self._last_prediction[k]
132             self.disturbance[k] = (1 - a) * self.disturbance[k] + a * err
133
134     # -----
135     def _candidate_grid(self, u_current: float) -> np.ndarray:
136         lo = max(self.p.u_min, u_current - self.p.max_move)
137         hi = min(self.p.u_max, u_current + self.p.max_move)

```

```

138 n = max(int(round((hi - lo) / self.p.candidate_step)) + 1, 1)
139 grid = np.linspace(lo, hi, n)
140 # always include "hold" exactly, useful for the infeasible fallback
141 if u_current not in grid:
142     grid = np.sort(np.append(grid, u_current))
143     return grid
144
145 def _violation(self, traj: Dict[str, np.ndarray]) -> float:
146     """Worst constraint violation (in psi, summed over pressure vars,
147     margin-inclusive) anywhere in the horizon. 0 if fully feasible."""
148     L = self.limits
149     m = self.p
150     viol = 0.0
151     viol += np.maximum(0.0, (L.whp_min + m.margin_whp) - traj["WHP"]).max()
152     viol += np.maximum(0.0, traj["WHP"] - (L.whp_max - m.margin_whp)).max()
153     viol += np.maximum(0.0, (L.flp_min + m.margin_flp) - traj["FLP"]).max()
154     viol += np.maximum(0.0, traj["FLP"] - (L.flp_max - m.margin_flp)).max()
155     viol += np.maximum(0.0, (L.bhp_min + m.margin_bhp) - traj["BHP"]).max()
156     viol += np.maximum(0.0, traj["BHP"] - (L.bhp_max - m.margin_bhp)).max()
157     return float(viol)
158
159 def _reject_reason(self, traj: Dict[str, np.ndarray]) -> Optional[str]:
160     L = self.limits
161     m = self.p
162     checks = [
163         ("BHP", traj["BHP"].min(), L.bhp_min + m.margin_bhp, "min"),
164         ("BHP", traj["BHP"].max(), L.bhp_max - m.margin_bhp, "max"),
165         ("WHP", traj["WHP"].min(), L.whp_min + m.margin_whp, "min"),
166         ("WHP", traj["WHP"].max(), L.whp_max - m.margin_whp, "max"),
167         ("FLP", traj["FLP"].min(), L.flp_min + m.margin_flp, "min"),
168         ("FLP", traj["FLP"].max(), L.flp_max - m.margin_flp, "max"),
169     ]
170     for name, val, bound, kind in checks:
171         if kind == "min" and val < bound:
172             k = int(np.argmin(traj[name]))
173             return f"{name} {val:.0f} < {bound:.0f} at k+{k+1}"
174         if kind == "max" and val > bound:
175             k = int(np.argmax(traj[name]))
176             return f"{name} {val:.0f} > {bound:.0f} at k+{k+1}"
177     return None
178
179 # -----
180 def step(
181     self,
182     measured: Dict[str, float],
183     u_current: float,
184     target_Q: float,
185 ) -> ControllerDecision:
186     """
187     Compute the next choke position.
188
189     Parameters
190     -----
191     measured : dict
192         Current {Q, WHP, FLP, BHP} measurement from the well.
193     u_current : float
194         Current choke position [%].
195     target_Q : float
196         Desired oil flow rate [bbl/hr].
197
198     Returns
199     -----
200     ControllerDecision with the selected choke position and the full
201     candidate evaluation (used for logging and for the dashboard's
202     reasoning panel).
203     """
204     # Filter first, then run the whole predict -> filter -> select cycle on
205     # the filtered state so the feasible set is stable interval to interval.
206     measured = self._filter_measurement(measured)
207     self._update_bias(measured)

```

```

208
209 grid = self._candidate_grid(u_current)
210 horizon_len = self.p.horizon
211 results: List[CandidateResult] = []
212
213 for u in grid:
214     u_seq = [u] * horizon_len # move-blocking: apply then hold
215     traj = self.model.predict(measured, u_seq, self.disturbance)
216     viol = self._violation(traj)
217     feasible = viol <= 1e-9
218     reason = None if feasible else self._reject_reason(traj)
219
220     q_end = float(traj["Q"][-1])
221
222     du = u - u_current
223     cost = (q_end - target_Q) ** 2 + self.p.move_penalty * du ** 2
224
225     results.append(
226         CandidateResult(
227             u=float(u),
228             du=float(du),
229             feasible=feasible,
230             reject_reason=reason,
231             predicted_Q_end=q_end,
232             predicted_trajectory=traj,
233             cost=float(cost),
234             max_violation=viol,
235         )
236     )
237
238     feasible_results = [r for r in results if r.feasible]
239     all_infeasible = len(feasible_results) == 0
240
241     if not all_infeasible:
242         # SELECT: minimum cost; ties -> smallest |move|; then lower u.
243         def sel_key(r: CandidateResult):
244             return (round(r.cost, 6), round(abs(r.du), 6), r.u)
245
246         chosen = min(feasible_results, key=sel_key)
247
248         # DEADBAND: only actually move if the move buys a meaningful
249         # predicted improvement over simply holding position. The cost
250         # surface is evaluated on noisy measurements, so without this its
251         # argmin wanders every interval and the choke chatters - pure valve
252         # wear with no production benefit. Holding is always preferred on
253         # a tie, which also makes the steady state genuinely steady.
254         hold = min(
255             (r for r in feasible_results if abs(r.du) < 1e-9),
256             key=lambda r: abs(r.du),
257             default=None,
258         )
259         if hold is not None and (hold.cost - chosen.cost) < self.p.min_cost_improvement:
260             chosen = hold
261         else:
262             # FALLBACK: minimise worst constraint violation; ties -> smallest
263             # |move| (never lurch), then lower u (favour closing).
264             def fb_key(r: CandidateResult):
265                 return (round(r.max_violation, 6), round(abs(r.du), 6), r.u)
266
267             chosen = min(results, key=fb_key)
268
269     selected_index = results.index(chosen)
270     self._last_prediction = {k: float(chosen.predicted_trajectory[k][0]) for k in OUTPUTS}
271
272     return ControllerDecision(
273         u_selected=chosen.u,
274         u_previous=u_current,
275         target_Q=target_Q,
276         candidates=results,

```

```
276 selected_index=selected_index,  
277 all_infeasible=all_infeasible,  
278 disturbance=dict(self.disturbance),  
279 )  
280
```